

Discussion 1 Version A Solutions

Questions

1. Given a list of unique integers, find the **second smallest** number.
 - 1.1 Describe the brute force algorithm. Explain why the brute force algorithm **is** indeed a brute force.

Iterate through each element and compare with every other element in the list. Return the number that is smaller than all other elements except one. Brute force, because it checks every possible comparison/possibility.
 - 1.2 What is its runtime?

$\Theta(n^2)$
 - 1.3 Come up with an optimal solution for the same problem.

Keep track of two numbers, the smallest and second smallest number you have seen as you iterate through the list. Update accordingly with every iteration. After iterating through the list, return the second smallest number.
 - 1.4 What is its runtime?

$\Theta(n)$

MOVE TO THE SIMILAR QUESTION IN VERSION B.

2. Give one disadvantage of timing programs to determine algorithm efficiency that is different from your partner. Explain in 2-3 sentences why it is a disadvantage.
 1. Time depends on the computer.

Different machines will run processes at different speeds, causing perceived differences in efficiency when algorithms might have the same asymptotic runtime. Additionally, the same machine may time the same program differently depending on other processes running at the time.
 2. Time depends on the particular input.

Different inputs can lead to very different execution times, even for the same algorithm. For example, some inputs may trigger worst-case behavior while others are processed much more quickly (ex. breaking early), making timing results inconsistent. As a result, a single timing measurement may not accurately reflect the algorithm's overall efficiency.
 3. One timing of an algorithm does not tell us how it scales.

Measuring runtime on a single input size is not a good indicator how the algorithm performs as the input grows. An algorithm that seems fast on small inputs may become much slower on larger ones, especially if it has poor asymptotic complexity. Without analyzing growth trends, timing alone can be misleading.

MOVE TO QUESTION 3 FOR VERSION A.

3. Using the code below:

3.1 Fill in the table.

Code	Runtime per line	# of executions
<pre>def count_positives(nums): count = 0 for x in nums: if x > 0: count += 1 return count</pre>	N/A	N/A
	c_1	1
	c_2	$n+1$
	c_3	n
	c_4	$\leq n$
	c_5	1

3.2 Solve for the runtime $T(n)$ of `count_positives`. Explain your work.

$$T(n) = c_1(1) + c_2(n+1) + c_3(n) + c_4(n) + c_5(1) = cn \rightarrow \Theta(n)$$

MOVE TO THE SIMILAR QUESTION FOR VERSION B.

4. What is the runtime of the code below. State your answer using Θ notation. *Make sure to explain the reason for your answer.*

```
def runtime_1(n):
    count = 0
    while n > 0:
        n = n // 2
        count += 1
    return count
```

$\Theta(\log(n))$. Each iteration halves n , so the number of iterations is the number of times you can divide n by 2 before reaching 0. This is proportional to $\log(n)$, so the runtime is $\Theta(\log(n))$. n is decreasing exponentially, which leads to a logarithmic number of iterations.

MOVE TO QUESTION 5 FOR VERSION A.

5. Given the code below, the runtime table has been filled out. However, there are **errors** in the table. Identify and fix the errors in the table. Once you have fixed the errors, calculate the runtime of `some_kind_of_counting` and express it in Θ notation.

Code	Runtime per line	# of executions	Line #
------	------------------	-----------------	--------

<pre>def some_kind_of_counting(n): count = 0 while n > 0: for i in range(n): count += 1 n = n - 1 return count</pre>	N/A	N/A	
	c_1	1	1
	c_2	$n + 1$	2
	c_3	$n^2 + 1 \rightarrow n(n+1)/2 + n$	3
	c_4	$n^2 \rightarrow n(n+1)/2$	4
	$n \rightarrow c_6$	n	5
	c_5	1	6

$$T(n) = c_1(1) + c_2(n+1) + \dots = cn^2 \rightarrow \Theta(n^2)$$

MOVE TO THE SIMILAR QUESTION IN VERSION B.

6. What is the complexity for the following expressions in terms of Θ ?

$$1 + 2 + 3 + 4 + 5 + \dots + (16n - 1) + 16n = ?$$

$\Theta(n^2)$, make sure they show their work

DSC 40B Discussion 1 Worksheet Version B

Questions

1. Given a list of integers, determine whether the list contains **duplicate integers**.

1.1 Describe the brute force algorithm. Explain why the brute force algorithm **is** indeed a brute force.

Check every possible pair of integers. If any contain the same integer, return true, otherwise return false. Brute force, because it checks every possible comparison.

1.2 What is its runtime?

$\Theta(n^2)$

1.3 Come up with an optimal solution for the same problem.

Initialize a dictionary to keep track of integers you have seen as you iterate through the list. If at an iteration the element is already in the dictionary, return true. If you iterate through the whole list, return false.

1.4 What is its runtime?

$\Theta(n)$

MOVE TO QUESTION 2 IN VERSION B.

2. Give one disadvantage of timing programs to determine algorithm efficiency that is different from your partner. Explain in 2-3 sentences why it is a disadvantage.

1. Time depends on the computer.

Different machines will run processes at different speeds, causing perceived differences in efficiency when algorithms might have the same asymptotic runtime. Additionally, the same machine may time the same program differently depending on other processes running at the time.

2. Time depends on the particular input.

Different inputs can lead to very different execution times, even for the same algorithm. For example, some inputs may trigger worst-case behavior while others are processed much more quickly (ex. breaking early), making timing results inconsistent. As a result, a single timing measurement may not accurately reflect the algorithm's overall efficiency.

3. One timing of an algorithm does not tell us how it scales.

Measuring runtime on a single input size is not a good indicator how the algorithm performs as the input grows. An algorithm that seems fast on small inputs may become much slower on larger ones, especially if it has poor asymptotic complexity. Without analyzing growth trends, timing alone can be misleading.

MOVE TO THE SIMILAR QUESTION IN VERSION A.

3. Using the code below:

3.1 Fill in the table.

Code	Runtime per line	# of executions
<pre>def count_odds(nums): count = 0 for x in nums: if x % 2 == 1: count += 1 return count</pre>	N/A	N/A
	c_1	1
	c_2	$n+1$
	c_3	n
	c_4	$\leq n$
	c_5	1

3.2 Solve for the runtime $T(n)$ of `count_odds`. Explain your work.

$$T(n) = c_1(1) + c_2(n+1) + c_3(n) + c_4(n) + c_5(1) = cn \rightarrow \Theta(n)$$

MOVE TO QUESTION 4 IN VERSION B.

4. What is the runtime of the code below. State your answer using Θ notation.

```
def runtime_2(n):
    counter = 1
    while counter < n:
        for i in range(n):
            print('I love puppies!')
        counter = counter * 2
    return "So cute!"
```

$\Theta(n \log(n))$. Each iteration halves n , so the number of iterations is the number of times you can divide n by 2 before reaching 0. This is proportional to $\log(n)$. At each iteration, the for loop runs for n iterations, so each iteration has $\Theta(n)$ runtime. Therefore the total runtime is $\Theta(n \log(n))$ as there are $\log(n)$ number of iterations.

MOVE TO THE SIMILAR QUESTION IN VERSION A.

5. Given the code below, the runtime table has been filled out. However, there are errors in the table. Identify and fix the errors in the table. Once you have fixed the errors, calculate the runtime of `some_kind_of_sum` and express it in Θ notation.

Code	Runtime per line	# of executions	Line #
def some_kind_of_sum(lst):	N/A	N/A	
count = 0	c_1	1	1
for i in range(len(lst)):	c_2	$n + 1$	2
for j in range(i):	c_3	$(n + 1)^2 \rightarrow n(n+1)/2 + n$	3
count += sum(lst)	$c_4 \rightarrow n$	$n^2 + n \rightarrow n(n+1)/2 + n$	4
if lst[i] == 0:	c_5	$n^2 + n \rightarrow n(n+1)/2 + n$	5
print('Zero!')	c_6	$n^2 + n \rightarrow \leq n(n+1)/2 + n$	6
return count	c_7	1	7

$$T(n) = c_1(1) + c_2(n+1) + c_3(n(n+1)/2 + n) + \dots = cn^3 \rightarrow \Theta(n^3)$$

MOVE TO QUESTION 6 IN VERSION B.

6. What is the complexity for the following expressions in terms of Θ ?

$$\log n + \log n^2 + \log n^3 + \log n^4 + \dots + \log n^n = ?$$

$\Theta(n^2 \log(n))$, make sure they show their work